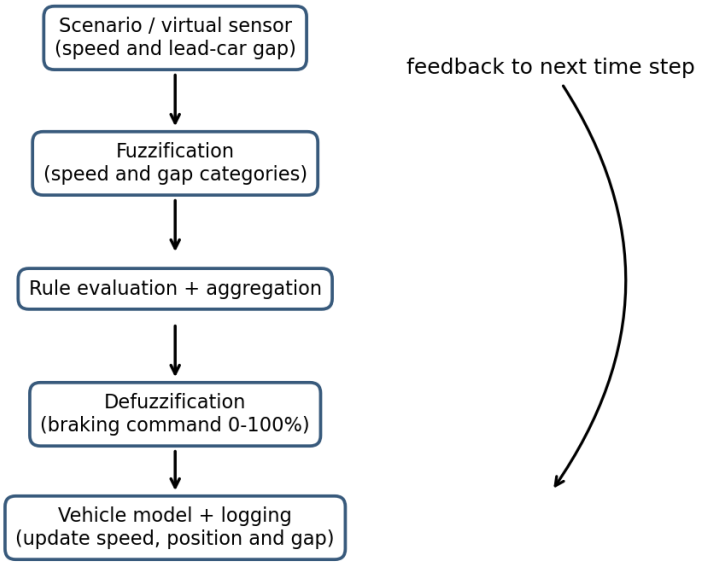
# FUZZY-LOGIC-BASED CAR SPEED CONTROL

## Front-Vehicle Detection and Distance-Aware Speed Regulation




## Portfolio Project Report

Software simulation project using Python, fuzzy rules, a simplified braking model, and graphical evaluation.

Prepared for: Mobashwer Hossain
Project type: Simulation and controller design
Primary tools: Python, NumPy, Matplotlib, optional scikit-fuzzy

Scope note: This version models following a vehicle ahead. It does not directly detect potholes and is not a road-ready autonomous-driving system.

# Abstract and Project Overview

This report describes a software simulation of a vehicle that adjusts its speed while following a lead vehicle. The central idea is to use a Fuzzy Logic Controller (FLC) to convert two measurable simulation inputs - the controlled vehicle's speed and the gap to the vehicle ahead - into a braking command. A simplified longitudinal vehicle model applies the command and updates the vehicle state over repeated time steps.

Fuzzy logic is useful when control decisions are naturally expressed in linguistic terms such as Very Near, Near, Medium, Far, Slow, Moderate, Fast, and High Braking. Instead of switching only at a single hard boundary, overlapping membership functions allow several rules to contribute to the output. The controller can therefore produce graded responses, provided that the rule base and membership functions are designed and tested carefully.

The project workflow includes defining the simulation scope, creating input membership functions, writing a rule base, computing a braking output, updating speed and gap, recording events, and plotting the resulting time series. The report also proposes evaluation against a simple threshold baseline and discusses limits such as sensor uncertainty, time-step size, actuator delay, and the absence of steering or lateral dynamics.

> The numerical examples and graphs in this report are illustrative. They explain how the implementation can be tested; they are not measured results from a real car. Any performance or safety claim must be supported by reproducible experiments.

Keywords: Fuzzy Logic Controller, car-following, speed regulation, braking model, membership functions, Python simulation, control evaluation.

# Contents and How to Use This Report


| Section | Topic | Purpose |
|---|---|---|
| 1 | Project definition and problem statement | Clarifies the task and motivation. |
| 2 | Objectives, scope, and assumptions | Defines what is and is not implemented. |
| 3 | System architecture and workflow | Shows modules and feedback flow. |
| 4 | Tools and software environment | Explains the role of each library. |
| 5 | Fuzzy logic fundamentals | Introduces linguistic sets and inference. |
| 6 | Membership-function design | Explains triangular sets and overlap. |
| 7 | Rule-base design | Provides an example rule matrix. |
| 8 | Braking model and vehicle equations | Connects command to speed updates. |
| 9 | Worked numerical example | Demonstrates one time-stepped calculation. |
| 10 | Algorithm and pseudocode | Gives the implementation sequence. |
| 11 | Python implementation plan | Outlines code modules and validation. |
| 12 | Simulation scenarios | Defines repeatable test cases. |
| 13 | Graphs and result interpretation | Explains useful plots. |
| 14 | Evaluation metrics | Defines measurable performance indicators. |
| 15 | Baseline comparison | Discusses threshold and PID alternatives. |
| 16 | Testing and edge cases | Covers failures and robustness. |
| 17 | Limitations and risk controls | Prevents overclaiming. |
| 18 | Future improvements | Lists practical extensions. |
| 19 | Portfolio presentation | Suggests how to publish the project. |
| 20 | Viva questions and answers | Supports project explanation. |
| 21 | Conclusion and references | Summarizes the project and source basis. |


Recommended reading path: read Sections 1-4 for the overall design, Sections 5-11 for the controller and implementation, then Sections 12-18 for testing and limitations. The viva section can be used to prepare a concise explanation for a project review.

# 1. Project Definition and Research Problem

## Proposed title: Fuzzy-Logic-Based Car Speed Control Using Front-Vehicle Gap in a Software Simulation.

The simulated controlled vehicle is often called the ego vehicle. A second vehicle travels ahead of it and acts as the lead vehicle. At each simulation step, the virtual sensor provides the ego speed and the gap between the two vehicles. The FLC uses these values to produce a braking request. The vehicle model applies the request, updates speed and position, and calculates the next gap.

A fixed-threshold controller can change its command abruptly when an input crosses a boundary. For example, it may switch from light braking to strong braking as soon as a gap falls below a chosen value. A fuzzy controller can express a gradual concept such as 'somewhat near' or 'very near' and combine it with the current speed category. This does not guarantee better performance, but it provides a clear design hypothesis that can be tested.

Research question: Under the same simulated scenarios, can a carefully tuned fuzzy controller regulate the following gap with acceptable speed response and fewer abrupt command changes than a simple threshold controller?

## Useful sub-questions

- How does braking change when the gap becomes shorter at the same ego speed?
- How does the same gap affect braking when ego speed is Slow versus Fast?
- Does the controller avoid negative speed and record minimum-gap violations?
- How sensitive are results to sensor noise, update interval, and membership-function parameters?

Important distinction: object detection and metric distance estimation are separate tasks. A camera bounding box alone does not reliably give distance in metres without calibration, depth information, or another ranging method. The first version should use a virtual sensor so the control logic can be validated independently.

# 2. Objectives, Scope, and Assumptions

## Project objectives

- Build a repeatable software simulation with an ego car, a lead car, and a virtual gap sensor.
- Define fuzzy membership functions for speed and gap, then calculate a braking command.
- Connect the controller to a simplified vehicle model and update speed and position over time.
- Add explicit stop logic and prevent negative speed values.
- Record speed, gap, braking command, stopping events, and minimum-gap violations.
- Plot time-series results and compare the controller with a baseline.

## In scope

Front-vehicle following in a one-dimensional simulation; virtual sensing; fuzzy inference; simplified longitudinal dynamics; logging; plotting; repeatable tests; and a written evaluation.

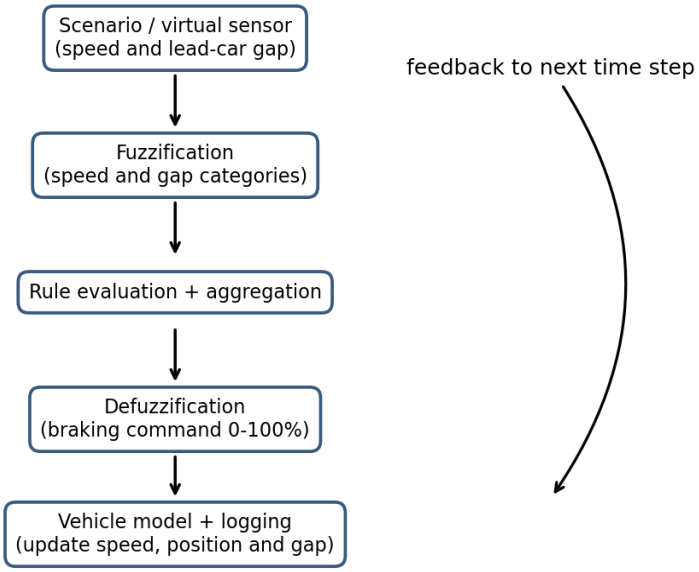
## Out of scope for the first version

Direct pothole detection, steering, lane keeping, lane changes, complete autonomous driving, a certified emergency-braking system, and public-road deployment are not part of the current simulation. Hardware may be considered only after software testing and safety review.

| Assumption | Why it matters |
|---|---|
| One-dimensional motion | Steering and lateral movement are not modelled. |
| Lead position and speed are known to simulator | No camera-based range claim is made in the baseline version. |
| Fixed time step | Large steps can cause inaccurate updates or overshoot. |
| Simplified braking relation | Real braking depends on vehicle mass, friction, road grade, and actuator response. |
| Speed is clamped at zero | The simulation must not represent reverse motion as a braking artefact. |
| Chosen target speed and minimum gap | These are simulation parameters, not universal safety thresholds. |

# 3. System Architecture and End-to-End Workflow

The project is organized as a feedback loop. The controller does not issue one command and stop; it repeatedly reads the updated state and reacts to it. This loop is the key to a meaningful simulation.




## Module responsibilities

| Module | Input / output | Responsibility |
|---|---|---|
| Scenario manager | Initial states and lead-car motion | Creates repeatable scenarios. |
| Virtual sensor | Ego speed and gap | Returns state values, optionally with noise. |
| Fuzzy controller | Speed + gap to braking command | Fuzzification, rule evaluation, aggregation, defuzzification. |
| Vehicle model | Braking command and current state | Updates speed and position. |
| Logger and evaluator | Time-series values and event flags | Stores data, plots curves, calculates metrics. |

End-to-end sequence: initialise the scenario; read and validate inputs; fuzzify speed and gap; evaluate the rules; defuzzify the output; limit the command; update the vehicle; recompute the gap; check stop and gap conditions; log the results; repeat until the simulation ends.

# 4. Tools and Software Environment

| Tool | Role in this project | Notes |
|---|---|---|
| Python | Controller, simulation loop, data logging | Readable and suitable for rapid prototyping. |
| NumPy | Arrays and numerical calculations | Supports repeatable calculations and efficient state updates. |
| Matplotlib | Graphs and visual analysis | Plots help inspect trends but do not prove safety. |
| scikit-fuzzy (optional) | Fuzzy membership and inference | Useful for a Mamdani-style controller; manual implementation can be easier to inspect. |
| Jupyter Notebook | Exploration and experiments | Useful for testing individual components and showing results. |
| VS Code | Project editing and debugging | Useful for organizing modules and running tests. |
| Git / GitHub | Version history and portfolio | Keep code, README, requirements, and sample plots together. |

## Minimum viable stack

Python, NumPy, and Matplotlib are enough to start the first simulation. Add scikit-fuzzy if you want a library-supported inference system. Keep the sensor virtual until the control loop, stopping logic, and evaluation tests behave consistently.

## Suggested folder structure

```
fuzzy-car-speed-control/
src/ - controller and vehicle model
tests/ - unit tests and edge cases
results/ - CSV logs and graphs
notebooks/ - experiments
README.md - setup, scope, sample output
requirements.txt - dependencies
```

Use a fixed random seed when testing sensor noise so that comparisons can be repeated. Record library versions and controller parameters alongside each experiment.

# 5. Fuzzy Logic Fundamentals

Classical control often uses numerical thresholds. Fuzzy logic instead allows a value to belong partly to more than one linguistic category. A gap of 28 metres, for example, might be partly Near and partly Medium. The membership degree is a number between 0 and 1; it is not a probability.

## The basic stages of a Mamdani-style fuzzy controller are:

- Fuzzification: map numeric inputs to membership degrees.
- Rule evaluation: calculate how strongly each IF-THEN rule is activated.
- Aggregation: combine the output fuzzy sets from activated rules.
- Defuzzification: convert the combined output set into one numeric braking command.

## Example linguistic variables

| Variable | Possible sets | Meaning |
|---|---|---|
| Gap d (metres) | Very Near, Near, Medium, Far | Distance from ego vehicle to lead vehicle. |
| Ego speed v | Slow, Moderate, Fast | Current speed of the controlled vehicle. |
| Braking B (%) | None, Low, Medium, High, Very High | Requested deceleration intensity, limited to 0-100%. |

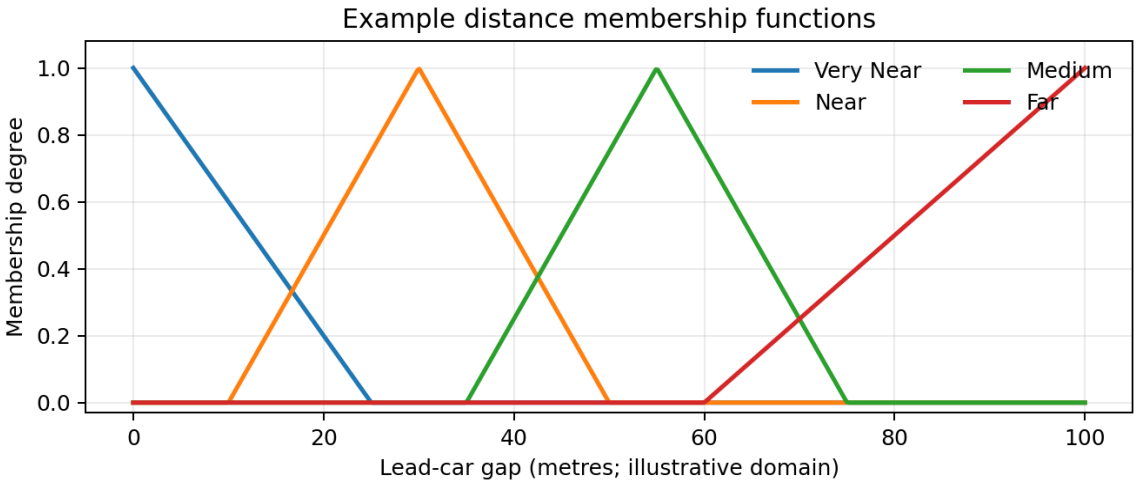
Example rule: IF the gap is Very Near AND ego speed is Fast, THEN braking is Very High. Another rule might say: IF the gap is Far AND ego speed is Slow, THEN braking is None or Low. The exact rule base must be checked for conflicts and uncovered combinations.

Fuzzy logic does not automatically make a controller safe or optimal. Results still depend on input ranges, membership shapes, rule choices, output scaling, and the accuracy of the simulated vehicle model.

# 6. Membership-Function Design

Triangular membership functions are a practical starting point because they are easy to understand and inexpensive to compute. A triangle is defined by three values: a, b, and c, where a is the left edge, b is the peak, and c is the right edge.

For a normal triangle, membership rises from zero at a to one at b, then falls to zero at c. At the domain boundaries, shoulder-shaped functions may be used so that the smallest and largest values remain represented.




*Illustrative distance membership functions. The numeric ranges shown are examples for demonstrating shape only; they must be tuned to the intended simulated speeds, gaps, and scenarios.*

## Design checks

- All expected input values should fall within the defined universe of discourse.
- Neighbouring sets should overlap enough that valid inputs do not activate zero rules.
- Endpoint cases must be handled without division by zero.
- The controller output should remain finite and within the selected limits.
- Plot each membership function before connecting it to the simulation.

When tuning, change one family of membership functions at a time and rerun the same scenarios. Keep the previous configuration so the effect of each change can be compared objectively.

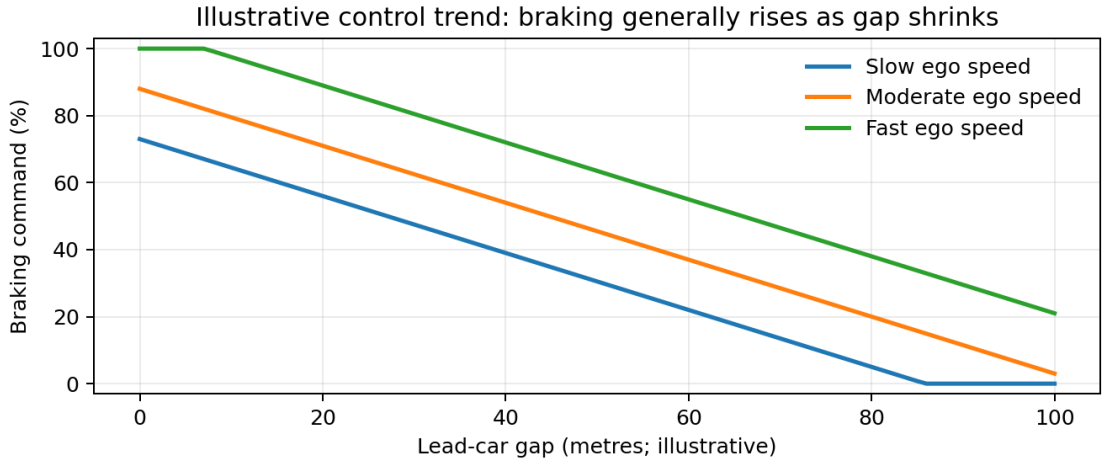
# 7. Rule-Base Design

A rule base converts qualitative driving behaviour into an explicit set of IF-THEN statements. For the two-input version, each rule combines one gap category with one ego-speed category. The following table is a starter example, not a validated driving policy.

| Gap \ Speed | Slow | Moderate | Fast |
|---|---|---|---|
| Very Near | High | Very High | Very High |
| Near | Medium | High | Very High |
| Medium | Low | Medium | High |
| Far | None | Low | Low |

## Rule interpretation

- A shorter gap generally increases the requested braking level.
- At the same gap, a faster ego vehicle generally receives a stronger braking request.
- A far gap and slow speed usually do not require strong braking.
- Rules should not be interpreted as a replacement for a certified safety controller.




*Illustrative trend only. The chart is a conceptual shape, not the output of a measured or validated controller.*

## Rule-base review checklist

Check all combinations, transitions near category boundaries, and cases where multiple rules activate together. Inspect whether a small input change causes an unexpectedly large output jump. Add relative speed as a future input if the project needs to distinguish closing distance from an increasing gap.

# 8. Fuzzy Inference and Defuzzification

For each simulation step, the numeric speed and gap are mapped to their membership degrees. Each rule then receives an activation strength. With the common minimum AND operator, the strength of a rule is the smaller of its two input membership degrees. A product operator is another possible design choice, but the choice should be documented.

After rule activation, output membership sets are clipped or scaled according to the selected inference method and then combined. The centroid method is a common way to convert the aggregated fuzzy set into a numeric command.

## Centroid defuzzification

**B* = integral[z x mu_agg(z)] dz / integral[mu_agg(z)] dz**

Here, mu_agg(z) is the aggregated membership degree at output value z. In a discrete implementation, approximate the integrals with sums over a grid of output values. If the denominator is zero, use an explicitly defined fallback rather than allowing a crash or a NaN value.

## Output constraints

- Clip the final braking request to the range 0-100%.
- Validate that the output is numeric and finite.
- Optionally limit how quickly the command can change between steps.
- Log the raw controller output as well as the command after limiting.

A command limiter can make the simulated response smoother, but it can also delay strong braking. If included, evaluate the trade-off instead of assuming that smoothing is always beneficial.

# 9. Vehicle and Braking-Response Model

The source implementation notes propose a simple relation in which braking percentage determines deceleration. This is useful for an introductory simulation, provided that it is described as a modelling assumption rather than a universal physical law.

## Simplified deceleration

$$a_{brake} = (B / 100) \times a_{max}$$

B is the braking command in percent and a_max is the selected maximum deceleration magnitude in metres per second squared. If B = 50 and a_max = 5 m/s^2, the model uses a deceleration magnitude of 2.5 m/s^2.

## Speed update

$$v_{next} = \max(0, v_{current} - a_{brake} \times \Delta t)$$

The max operation prevents a negative speed. The time step delta_t is measured in seconds. Smaller time steps usually provide better temporal resolution, but they require more simulation iterations.

## Position and gap update

$$x_{ego,next} = x_{ego,current} + v_{ego,current} \times \Delta t$$

$$gap_{next} = x_{lead,next} - x_{ego,next} - L_{lead}$$

Use a consistent coordinate convention. L_lead is the effective length of the lead vehicle when positions refer to vehicle reference points. For improved numerical accuracy, position may be updated using the average of current and next speed.

## Limitations of the simplified model

The model does not capture tyre-road friction, road grade, brake temperature, actuator delay, vehicle mass variation, tyre slip, or real brake dynamics. Do not present its output as real stopping-distance evidence.

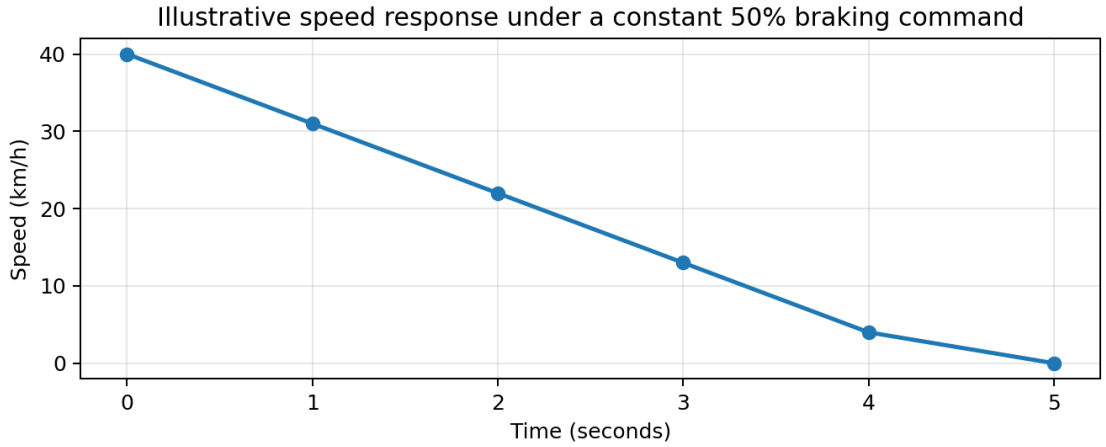
# 10. Worked Numerical Example

The handwritten implementation notes include a simple example with initial speed 40 km/h, braking command 50%, maximum deceleration 5 m/s^2, and a time step of one second. This example illustrates the arithmetic used by the simplified model.

| Parameter | Value |
|---|---|
| Initial speed | 40 km/h = approximately 11.11 m/s |
| Braking command B | 50% |
| Maximum deceleration a_max | 5 m/s^2 |
| Time step delta_t | 1 second |
| Modelled deceleration | (50 / 100) x 5 = 2.5 m/s^2 |

Applying v_next = max(0, v_current - a_brake x delta_t) gives the following approximate sequence:

| Time (s) | Speed (m/s) | Speed (km/h, approx.) |
|---|---|---|
| 0 | 11.11 | 40.0 |
| 1 | 8.61 | 31.0 |
| 2 | 6.11 | 22.0 |
| 3 | 3.61 | 13.0 |
| 4 | 1.11 | 4.0 |
| 5 | 0.00 | 0.0 |




*Illustrative speed sequence based on the simplified calculation. The one-second time step is coarse and should not be treated as a realistic braking experiment. For finer response, test smaller steps such as 0.05-0.1 seconds and perform a time-step sensitivity check.*

# 11. Algorithm and Pseudocode

The recommended implementation order separates controller development from vehicle simulation. This makes it easier to find bugs and verify each module before integrating the complete loop.

| Phase | Task | Acceptance check |
|---|---|---|
| 1 | Set up Python and libraries. | A minimal script runs. |
| 2 | Define units and input ranges. | All variables have documented units. |
| 3 | Build and plot membership functions. | No unintended gaps or endpoint errors. |
| 4 | Create rules for each speed-gap combination. | Every test input activates at least one rule. |
| 5 | Implement inference and defuzzification. | Output stays finite and between 0 and 100. |
| 6 | Build ego/lead vehicle state updates. | Updates match hand calculations. |
| 7 | Connect FLC to simulation loop. | Commands affect speed as expected. |
| 8 | Add stopping logic and event logging. | No negative speed; events are recorded. |
| 9 | Run scenarios and plot results. | Experiments are repeatable. |
| 10 | Tune and rerun tests. | Changes are justified by metrics. |

## Pseudocode

```
initialise ego state, lead-car state, controller, and log
for each time step:
read ego speed and lead-car gap
validate and bound sensor inputs
fuzzify speed and gap
evaluate fuzzy rules and aggregate outputs
defuzzify to braking command B
apply command limits
compute deceleration and update ego speed/position
update lead-car position and recompute gap
check stop state and gap violations
log time, speed, gap, command, and event flags
plot time series and calculate metrics
```

# 12. Python Implementation Plan

A maintainable implementation can be split into small functions rather than one long script. Each function should have a clear input, output, and test.

| Function / module | Responsibility | Example tests |
|---|---|---|
| membership.py | Membership functions and input sets | Boundary values, overlap, valid range. |
| controller.py | Rule evaluation and defuzzification | Known inputs, output bounds, zero denominator. |
| vehicle.py | Speed, position, and gap updates | Hand-calculated one-step update. |
| scenario.py | Lead-car motion and initial conditions | Repeatability and scenario validity. |
| logger.py | CSV output and event flags | Column names, row count, no missing critical values. |
| run_simulation.py | Orchestrate the feedback loop | End-to-end completion and deterministic run. |
| plot_results.py | Create graphs from saved logs | Labels, units, and file creation. |

## Recommended validation checks

- Input speed and gap are numeric, finite, and within the declared range.
- Braking command is always between 0 and 100%.
- Speed never becomes negative.
- The gap is calculated using the same coordinate convention throughout.
- The simulation stops at its time limit or a defined terminal condition.
- The same configuration and random seed reproduce the same output.

For portfolio use, include a requirements file, a short setup guide, screenshots of the graphs, and a small example CSV. Never publish generated example values as measured real-world performance.

# 13. Simulation Scenarios and Test Plan

A single smooth scenario is not enough to evaluate a controller. Build several cases that exercise ordinary operation and difficult transitions. Use the same starting conditions when comparing different controllers.

| Scenario | Lead-car behaviour | What to inspect |
|---|---|---|
| A. Steady lead | Lead car maintains near-constant speed. | Settling behaviour and stable following gap. |
| B. Lead slows down | Lead car reduces speed gradually. | Braking timing, minimum gap, command smoothness. |
| C. Sudden slowing | Lead car decelerates quickly. | Worst-case gap and response delay. |
| D. Short initial gap | Vehicles start closer than normal. | Strong braking and stop logic. |
| E. Sensor noise | Small random errors affect measured gap. | Command oscillation and robustness. |
| F. Delayed sensing | Gap measurement is delayed by a fixed amount. | Sensitivity to stale input. |
| G. Lead stops | Lead car reaches zero speed. | Whether ego vehicle also stops without negative speed. |
| H. Gap increases | Lead car accelerates or pulls away. | Whether unnecessary braking decreases. |

For each case, store the initial conditions, controller version, time step, membership parameters, rule table, random seed, and run date. This makes it possible to reproduce a surprising result and explain why two versions differ.

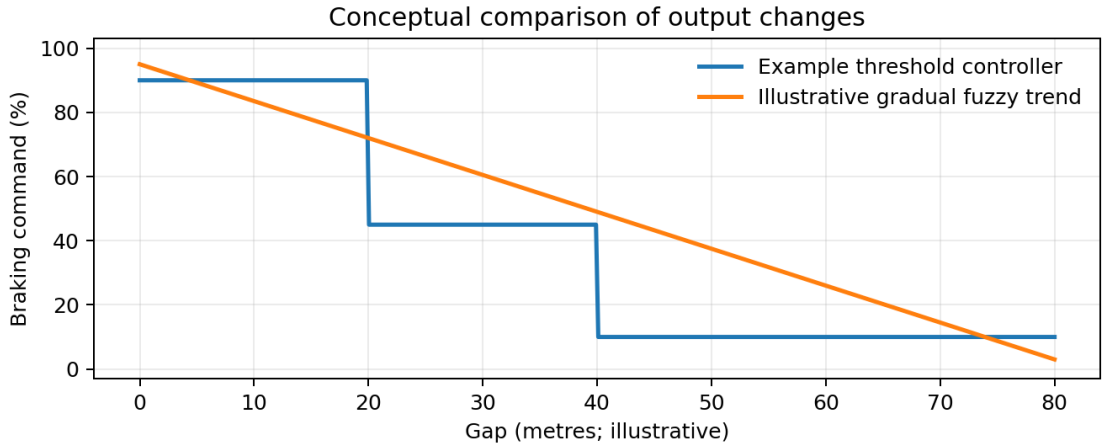
Do not hide collisions by forcing a negative gap back to zero. Record the violation explicitly so that the evaluation exposes failure rather than disguising it.

# 14. Graphs and Result Interpretation

Graphs should explain the behaviour of the controller, not merely decorate the report. Use consistent units and clear titles. The three most useful core plots are speed versus time, gap versus time, and braking command versus time.

## Recommended plots

- Speed versus time: shows deceleration, stopping, and speed recovery.
- Gap versus time: shows whether the vehicle maintains a suitable simulated following gap.
- Braking command versus time: shows how strongly and how often the controller reacts.
- Membership functions: shows how numerical values map into linguistic categories.
- Controller comparison: shows the same scenario under FLC and baseline logic.




*Conceptual comparison only. It illustrates how a threshold controller may change output in steps while a gradual mapping can vary smoothly; it is not a measured result from the implemented controller.*

## Interpretation questions

Does braking begin early enough in the tested case? Is the command noisy? Does the minimum gap become negative? Is the vehicle able to stop without numerical overshoot? Are differences between controllers repeatable across scenarios? Answer these questions using saved logs and explicit metrics.

# 15. Evaluation Metrics

Quantitative metrics help compare controller versions objectively. No single metric fully describes following behaviour, so report several complementary measures and always include the scenario definition.

| Metric | Definition / use | Caution |
|---|---|---|
| Minimum gap | Smallest gap recorded during a run. | A negative value indicates a simulated overlap/collision event. |
| Gap violation count | Number of time steps below the chosen minimum-gap threshold. | Threshold is a project parameter, not a universal safe distance. |
| Stopping time | Time from a defined event until speed reaches zero. | Depends on the chosen initial speed and simplified braking model. |
| Peak braking command | Maximum command value in a run. | High command alone does not prove a good response. |
| Command variation | Sum or average of absolute command changes. | A smoother command may still be too slow. |
| Speed tracking error | Difference between actual simulated speed and target speed. | Only meaningful when a target speed is defined. |
| Computation time | Runtime per simulation or per time step. | Measure on the same machine and setup. |

## Example reporting format

For each scenario, report minimum gap, number of gap violations, stopping time where applicable, peak braking command, command variation, and runtime. Include the configuration used and identify any failed run. If you have not run the experiment, label the table as a planned evaluation rather than filling it with invented measurements.

# 16. Baseline Comparison: Threshold, Fuzzy, and PID

A fuzzy controller should be compared against a baseline under identical initial conditions and lead-car motion. A simple threshold controller is easy to implement and provides a useful starting point. A PID controller may also be evaluated, but it requires a defined error signal and appropriate tuning.

| Approach | Strength | Trade-off |
|---|---|---|
| Threshold logic | Simple, transparent, fast to implement. | Can change commands abruptly around thresholds. |
| Fuzzy logic | Rules are interpretable and linguistic categories can overlap. | Performance depends on rule design and membership tuning. |
| PID | Well-known feedback approach with systematic analysis tools. | Needs a suitable error signal and tuning; saturation and delay matter. |
| Learned controller | Can represent complex patterns when data and training are adequate. | Needs representative data, validation, explainability, and safety constraints. |

## Fair-comparison procedure

- Use the same initial ego speed, initial gap, lead-car trajectory, and simulation duration.
- Use the same time step and sensor-noise sequence.
- Define common metrics before looking at results.
- Run multiple scenarios and, where noise is random, multiple seeds.
- Report failures and adverse cases, not only the best-looking graph.

The purpose of comparison is not to prove that fuzzy logic is always better. It is to identify the conditions under which a particular implementation behaves well or poorly.

# 17. Testing, Edge Cases, and Error Handling

Edge cases are important because a controller can appear to work in a normal scenario while failing at boundaries or invalid inputs. Test these cases deliberately.

| Case | Expected handling |
|---|---|
| Zero ego speed | Maintain a stopped state unless the model explicitly supports acceleration. |
| Zero or negative measured gap | Record a severe gap violation and invoke the chosen emergency simulation logic. |
| Very large gap | Keep output within limits and avoid invalid membership values. |
| NaN or infinite input | Reject or replace using an explicit, logged fallback policy. |
| No rule activated | Use a documented fallback output and record the event. |
| Zero centroid denominator | Avoid division by zero and log the fallback. |
| Very large time step | Warn or reject it; test time-step sensitivity. |
| Lead vehicle suddenly stops | Check response delay, minimum gap, and collision flags. |
| Output oscillation | Inspect overlapping sets, noisy inputs, and command-rate limits. |

## Testing layers

Unit tests: validate membership functions, inference, and state updates separately. Integration tests: connect the controller and vehicle model. Scenario tests: run the full loop under different lead-car behaviours. Regression tests: rerun saved scenarios after changes to ensure a fix does not create a new failure.

A test passes only when the expected behaviour is checked explicitly. A script that runs without crashing is not sufficient evidence that the controller is correct.

# 18. Limitations and Safety Considerations

This project is a software simulation and learning exercise. It is not a certified driver-assistance feature, and it should not be connected to a real vehicle for road use. A simple model cannot represent all the physical and environmental factors that determine stopping behaviour.

## Main limitations

- No steering, lane changing, road curvature, or lateral vehicle dynamics.
- Simplified braking dynamics without validated tyre-road friction or actuator response.
- The virtual sensor may provide cleaner information than real sensors.
- Camera detection, if added, does not by itself provide accurate distance in metres.
- The two-input controller does not directly account for relative speed unless that input is added.
- Membership functions and rules are hand-designed and may not generalize to every scenario.
- Plots and simulated metrics cannot establish real-world safety.

## Risk controls for the software project

- Label every simulated graph and example clearly.
- Log collision or gap violations instead of concealing them.
- Use a conservative, explicit fallback for invalid controller outputs.
- Keep physical hardware out of the first version.
- If a physical prototype is ever considered, use a small test platform, an independent emergency stop, guarded motors, and supervised low-speed testing in a controlled area.

Do not claim 'prevents accidents' or 'guarantees safe braking' unless there is appropriate evidence and validation. Prefer precise wording such as 'evaluated in a simplified simulation under the listed scenarios.'

# 19. Future Improvements and Research Extensions

Once the baseline simulation is stable, extensions can be introduced one at a time. Each extension should have a clear research question and a measurable evaluation plan.

| Extension | What it adds | How to evaluate it |
|---|---|---|
| Relative speed input | Distinguishes closing gap from a gap that is increasing. | Compare minimum gap and braking onset across matched scenarios. |
| Sensor noise and delay | Makes virtual sensing less ideal. | Sweep noise level and delay; measure violation rates. |
| Adaptive time gap | Sets desired following distance based on speed. | Evaluate gap tracking and transient response. |
| PID baseline | Provides a common feedback-control comparison. | Tune fairly and compare identical scenarios. |
| Better vehicle dynamics | Adds acceleration limits and actuator lag. | Check sensitivity to model parameters. |
| YOLO front-car detection | Detects a vehicle in camera frames. | Measure detection quality separately from distance accuracy. |
| Depth/ranging sensor | Provides a route to metric distance measurement. | Calibrate and quantify measurement error. |
| Small RC prototype | Demonstrates the control loop physically. | Use controlled conditions and independent emergency stop. |

A useful extension is to add relative speed, delta-v = ego speed minus lead speed. A vehicle closing quickly may require more braking than a vehicle at the same gap whose gap is increasing. This addition also makes the assumptions of the original two-input controller more visible.

Treat every new sensor or model as a separate subsystem. Validate its output before relying on it in the controller.

# Conclusion, References, and Project Status

## Conclusion

This project provides a structured way to study fuzzy-logic control for vehicle following. A virtual sensor supplies the ego speed and lead-car gap, a fuzzy inference system produces a braking command, and a simplified vehicle model updates speed and position. The design can be evaluated using repeatable scenarios, graphs, and quantitative metrics.

The most important next step is implementation and testing: verify the membership functions, cover the rule combinations, validate the speed update, log unsafe gap events, and compare the FLC with a baseline using identical scenarios. The report's example graphs and calculations are explanatory only and should not be presented as measured project results.

## References and source basis

1. Fuzzy-Logic-Based Car Speed Control: Front-Car Detection and Distance-Aware Speed Regulation, uploaded project report (19 pages). Used for the proposed simulation scope, controller stages, braking equations, algorithm plan, and evaluation guidance.

2. IMPLEMENTATION IDEA, uploaded handwritten notes (5 pages). Used for the step-by-step implementation plan, architecture sketch, and the illustrative braking calculation beginning at 40 km/h with a 50% command.

3. Ahmed S. AlGhamdi and Halit Eren, “Using FUZZY Logic to Control Vehicles' Velocity in Traffic System,” IFAC Intelligent Autonomous Vehicles Symposium, Gold Coast, Australia, June 26-28, 2013. Used as background reading on fuzzy control and vehicle-following systems.

## Project status note

This report is a design and implementation guide. Before uploading it as a completed project, confirm which code modules have actually been implemented and replace illustrative figures with plots generated by your own saved simulation logs. Keep the scope statement visible in the portfolio.